Université de Bordeaux
Master 1 Informatique
Projet de Programmation
Scrabble en Java

Alexandre Lou-Poueyou
Aymeric Cassard
Ludovic Andrieu
Jean Mayol
Mathéo Petry
Valentin Cazet

10 avril 2025

Table des matières

1 Règles du jeu [2] **3**
1.1 Installation de la partie 3
1.2 Déroulement de la partie 3
1.3 Fin de la partie 4
1.4 Comptage des points 4
1.5 Conseil de jeu 4
1.6 Variantes du jeu 5
1.7 Exemple début de déroulement d'une partie 5
1.8 Complexité 7
1.9 Stratégies 7

2 L'existant **7**
2.1 Histoire 7
2.2 Contexte général 8
2.3 Existant du projet 8

3 Structures de données & algorithmes **10**
3.1 Structures de données 10
3.1.1 `Dictionary.java` 10
3.1.2 `Letter.java` 11
3.1.3 `LetterCounter.java` 11
3.1.4 `Grid.java` 12
3.1.5 `History.java` et `HistNode.java` 12
3.2 Algorithmes 14
3.2.1 `Parser` 14
3.2.2 `Tokens` 14
3.2.3 `Serializer` 17
3.2.4 `placeLetters()` 18

4 Architecture **26**
4.1 Architecture 26
4.1.1 `ViewCLI` sur MVP 27
4.1.2 `ViewGUI` sur VM 29
4.2 Gestion des erreurs 31
4.3 Dépendances du projet 32

5 Performances et limitations **32**
5.1 Choix de conception et limitations observées 32
5.2 Analyse des performances 34

6 Critiques et perspectives **35**

7 Bibliographie **36**

A Besoins réalisés **37**
A.1 Liste des besoins réalisés 37
A.2 Note 38

1 Règles du jeu [2]

Le Scrabble est un jeu de société se jouant de 2 à 4 joueurs et qui consiste à former des mots sur une grille carrée de 15×15 cases. Le but du jeu est de marquer le plus de points en posant des mots. Le jeu du Scrabble contient un plateau de jeu, un ensemble de jetons de lettres variant selon le pays, un sac de lettres et des chevalets de jeu permettant de poser les jetons tout en cachant ses lettres aux adversaires. En français il y a 102 jetons en comptant les 2 jokers par exemple. En anglais il y en a 100. Chaque lettre a un certain score qui lui est attribué. Plus il est difficile de faire un mot avec une lettre, plus le score de cette lettre est élevé. Par exemple, la lettre `A` vaut 1 tandis que le `W` vaut 10 points. Les jokers ne valent pas de points.

1.1 Installation de la partie

On met tous les jetons de scrabble dans le sac de lettres. On pose au centre de la table le plateau de jeu et devant chaque joueur des chevalets de jeu. Chaque joueur pioche sans regarder une lettre dans le sac de lettres. Celui qui a la lettre la plus haute dans l'alphabet (A > B, B > C, etc.) commence. Si un joueur pioche le joker, il doit re-piocher. Ils remettent toutes les lettres quand le 1er joueur est déterminé.
Le 1er joueur pioche ensuite 7 lettres dans le sac de lettres et les garde sans les montrer aux adversaires. Les autres joueurs font de même dans le sens horaire.
La partie peut désormais commencer.

1.2 Déroulement de la partie

Au début de la partie, le 1er joueur pose sur le plateau un mot qu'il peut former à partir des lettres de son chevalet. Une lettre du mot doit être posée sur le centre du plateau (la case de coordonnée $h8$ pour un plateau standard).Il compte le nombre de points réalisés avec ce mot puis il pioche dans le sac de lettres un nombre de lettres égal au nombre de lettres posées sur le plateau. Il pioche donc jusqu'à avoir 7 lettres sur son chevalet.
Le 2ème joueur fait de même mais doit rattacher son mot au mot du 1er joueur et ne peut superposer ces lettres sur le mot du 1er joueur. Il faut que le mot rattaché au mot du 1er joueur soit aussi un mot légal, c'est-à-dire un mot appartenant au dictionnaire. Ensuite, il score et pioche, suivant la même règle que celle du 1er joueur. La partie se déroule ainsi de suite jusqu'à la fin.

Les seules actions que peut entreprendre le joueur lors de son tour sont :

— Poser un mot.

— Échanger des lettres avec la pioche.

— Passer son tour.

1.3 Fin de la partie

Les conditions de fin de partie sont les suivantes :

— **Reliquat épuisé** : Un joueur a épuisé toutes ses lettres et la pioche est vide. Les joueurs qui n'ont pas terminés retirent de leurs scores la valeur des lettres qu'ils n'ont pas posés et le joueur qui a finit ajoute a son score l'ensemble de ces valeurs ainsi retirés.

— **Blocage** : Les joueurs ne trouvent plus de mots a jouer et il reste moins de 7 lettres dans la pioche. Les valeurs des lettres de leur mains sont déduites de leur score final.

Le joueur avec le score le plus élevé gagne. En cas d'égalité, les joueurs sont déclarés ex aequo.

1.4 Comptage des points

La grille contient des cases bonus :

— Lettre double : double la valeur de la lettre posée.

— Lettre triple : triple la valeur de la lettre posée.

— Mot double : double la valeur du mot posé.

— Mot triple : triple la valeur du mot posé.

Le score du coup posé est la somme des valeurs des lettres de chaque mot formé par le coup, majorée des points obtenus en plaçant des lettres sur des cases multiplicatrices. On applique les bonus de lettres avant d'appliquer celui des mots. Si plusieurs mots sont formés occupant la même case multiplicatrice d'un seul coup, le joueur obtient le bonus correspondant à chaque mot formé. Une fois qu'une case multiplicatrice a été utilisée, elle ne rapporte plus de points si un autre mot est formé ultérieurement avec la tuile qui l'occupe.

Si un joueur pose toutes ses lettres en un tour, il réalise un "scrabble" et obtient un bonus de 50 points.

1.5 Conseil de jeu

Si vous pensez que vous pouvez bientôt réaliser un scrabble, n'hésitez pas à garder les lettres qui permettent cela. Jouer les lettres qui rapportent sur les cases

lettres doubles ou triples. La lettre 's' est stratégique en français car elle peut s'insérer dans plein de mots pour les terminaisons.

1.6 Variantes du jeu

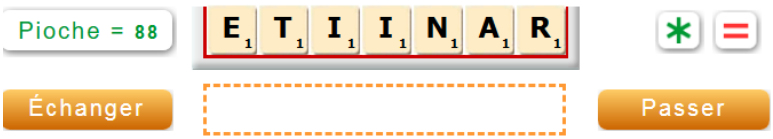
Il existe plusieurs variantes au scrabble, dont :

— **Super-scrabble** :
grille de taille 21×21
la pioche contient plus de lettres
chaque lettres a un multiplicateurs de score différents

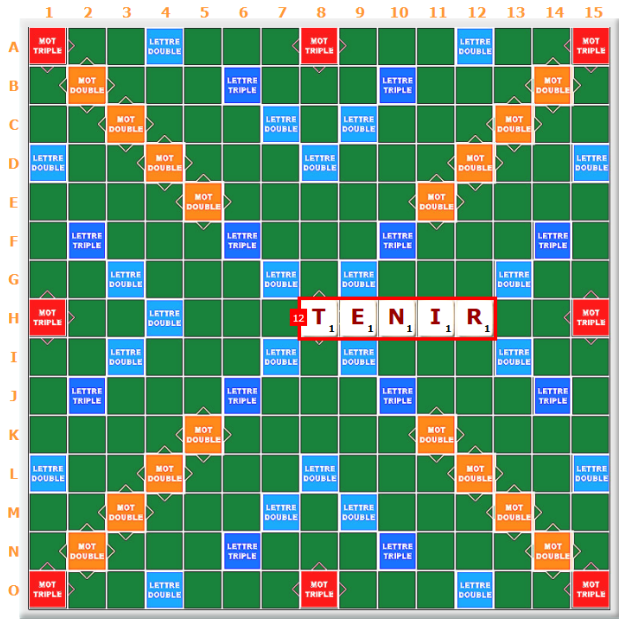
— **Blitz** :
chaque joueur dispose d'un temps imparti
ce temps s'écoule durant son tous
un joueur n'ayant plus de temps perd la partie

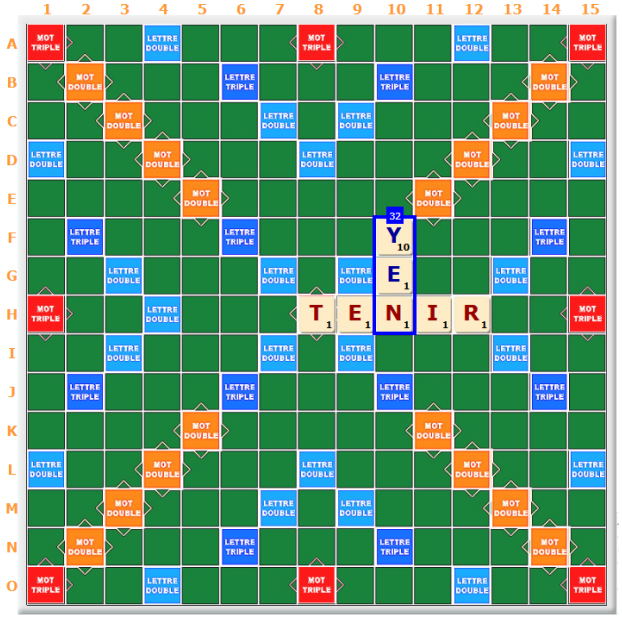
1.7 Exemple début de déroulement d'une partie

Pour l'exemple, on vous propose le début d'une partie contre une IA sur un site de jeu de scrabble en ligne. [8]
Voici une main de départ classique d'un joueur :



On a trouvé et choisi de placer le mot `TENIR` au centre du plateau ci -dessous 1.7. Chaque lettre nous donne un point en comptant qu'on a posé le 'R' sur une lettre compte double et le 'E' sur un mot compte double, notre score est donc de : $(1 + 1 + 1 + 1 + 1 \times 2) \times 2 = 12$ points.





Le mot joué par l'ordinateur est le mot `YEN` se rattachant à notre mot comme il se doit. L'adversaire compte donc le mot qu'il vient de former, sachant qu'il a placé un `Y` sur une case lettre compte triple. Ce qui donne comme score : $10 \times 3 + 1 + 1 = 32$.
Ce qui fait un score de 12 contre 32. Les joueurs continuent donc de jouer ainsi

jusqu'à ce qu'il n'y ait plus de mot dans le sac de lettres et qu'un des joueurs n'ait plus de lettres aussi.

1.8 Complexité

Il s'agit d'un jeu de société qui allie stratégie, vocabulaire et calcul. La complexité réside dans la nécessité de former des mots tout ayant le plus grand score, en utilisant les lettres disponibles et en tenant compte des cases spéciales sur le plateau telles que les doubles mots ou doubles lettres. Les joueurs doivent anticiper les mouvements des autres joueurs, bloquer leurs opportunités et gérer leur propre sac de lettres. De plus, la connaissance des mots peu courants et des stratégies de placement peut faire la différence entre la victoire et la défaite.

1.9 Stratégies

Privilégiez l'utilisation des cases bonus, comme les cases double ou triple lettre et mot, pour maximiser les points. Ensuite, essayez de former des mots parallèles, en ajoutant des lettres à des mots déjà présents sur le plateau pour marquer des points supplémentaires. Gardez un œil sur les lettres des autres joueurs et bloquez leurs opportunités en plaçant des mots stratégiques. Enfin, apprenez quelques mots courts et peu courants, car ils peuvent vous aider à utiliser des lettres difficiles et à créer des opportunités de score.

2 L'existant

2.1 Histoire

En 1931, l'ancêtre du scrabble [3], le 'Lexico' fut créé par un jeune architecte Américain Alfred Mosher Butts : chaque joueur pioche 7 lettres et peut changer leurs lettres durant leurs tours de jeu. Dès qu'un joueur trouve un mot avec 7 lettres, il gagne. Après certains avis constructifs des joueurs, Alfred Butts continue d'améliorer petit à petit son jeu. C'est à l'aide de James Brunot, un homme d'affaire que le jeu va être repris, modifier et devenir le scrabble que l'on connaît. Il est officiellement crée le 16 décembre 1948 sous ce nom, signifiant en anglais "chercher à tâtons". C'est à partir des années 50 que le scrabble va se propager dans le monde et que des fédérations vont voir le jour.

2.2 Contexte général

Le jeu du scrabble est un jeu populaire se jouant généralement en famille, il existe des compétitions de scrabble officiels et un classement pour les joueurs compétitifs. Le jeu du scrabble est connu pour sa simplicité des règles mais aussi pour sa complexité et recherche permanente de mots a trouver. La profondeur du jeu est multiple : il faut s'adapter aux mots posés. Cela augmente au fur et à mesure les chances de trouver un mot avec ses propres lettres. De plus, il faut prendre en compte l'aléatoire de la pioche de lettres qui rend compliquée l'implémentation d'une IA qui doit tout gagner.

2.3 Existant du projet

Le scrabble étant un jeu très connu et répandu, il n'est pas étonnant de trouver des existants de ce jeu, principalement pour ce qui est du test de performances d'algorithmes. On peut retrouver dans cette situation le Quackle [7] 1, un projet développé par Jazon Katz-Brown, John Fultz, John O'Laughlin et Anand Buddhev déposé sous licence GPL (General Public Licence) version 2 permettant la consultation, modification et redistribution du code cependant celui ci doit reste sous licence GPL version 2. Il a été conçu pour jouer contre une IA puissante, étudier des parties jouées et analyser les coups effectués, son IA est fait à base d'heuristique utilisant le Gaddag [6], une structure de données (présentée par Steven Gordon en 1994) utilisée pour la recherche de mots dans le Scrabble ou tout autre jeu nécessitant une génération de mots.

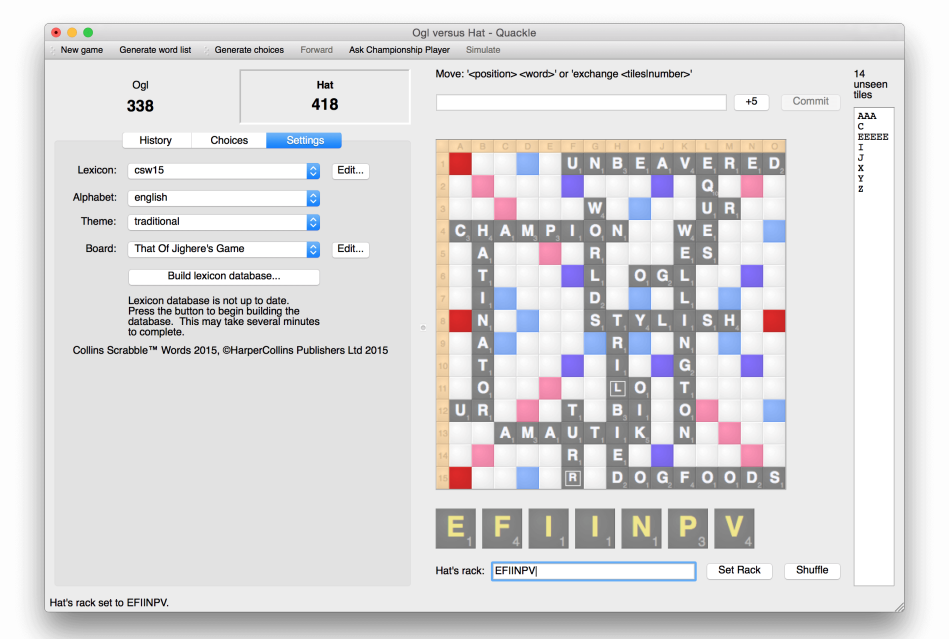


FIGURE 1 – Quackle sur macOS

Mais on peut aussi retrouver des implémentations du Scrabble se concentrant sur l'aspect ludique et distrayant via l'ajout de différents modes de jeu, d'un aspect compétitif ou d'éléments de personnalisation. Par exemple, les jeux de Scrabble développés pour les mobiles au travers de l'application "Scrabble GO" développée par Scopely pour iOS et Android, avec l'aval de Hasbro et Mattel bénéficiant donc d'une licence officielle. Celle-ci offre à l'utilisateur le moyen de jouer en ligne avec ses amis ou des inconnus, mais aussi d'aborder le Scrabble sous un œil différent via des modes de jeu inédits comme le *Tumbler* étant un mode jeu en solo où le joueur doit trouver le maximum de mots à fort score dans un temps limité, ou encore le *Rush* qui propose une partie sur un plateau plus petit (11x11) dans un temps et un nombre de tours limités.



FIGURE 2 – Scrabble Go sur mobile

De multiples Scrabble en ligne ont été développés et certains sont partagés par la Fédération Internationale de Scrabble Francophone [4]. D'autres exemples open source sont disponibles en ligne sur GitHub, tel que Scrabble-Java-FX par Typhon0 implémentant un Scrabble en JavaFX. Il existe d'autres programmes permettant de trouver des mots pour le Scrabble tel que WordFinder par jchecca973.

3 Structures de données & algorithmes

3.1 Structures de données

3.1.1 `Dictionary.java`

La classe dictionnaire permet de stocker l'ensemble des mots autorisés pour une partie de Scrabble. Un dictionnaire est un ensemble de mots, un mot existant en un seul exemplaire. Cette classe est implémentée grâce à un `HashSet<Word>`. Il s'agit d'un type permettant de créer un ensemble mathématique de Word, c'est-à-dire de mots. Lors d'une partie, il est souvent nécessaire de savoir si un mot appartient au dictionnaire, pour pouvoir vérifier qu'un joueur place un mot existant, un mot dit légal. De ce fait, l'ensemble de mots est de type `HashSet<Word>` car ce type d'ensemble est implémenté comme une table de hachage, ce qui permet de vérifier l'existence d'un mot dans le dictionnaire en une complexité $\mathcal{O}(1)$.

Cette classe est implémentée comme ci-dessous. On remarque un attribut supplémentaire `cachedDicts` servant de cache pour accélérer les tests, jusqu'à 10 fois moins de temps pour effectuer les tests :

```
1 public class Dictionary implements IDictionary {
2     private final Set<Word> words = new HashSet<>();
3
```

```
4       /** for fast tests */
5       private static final HashMap<Path, Set<Word>>
            cachedDicts = new HashMap<>();
6   }
```

La classe `Dictionary` implémente l'interface `IDictionary`, pour pouvoir changer la structure de données du dictionnaire par le `Dawg` ou le `Gaddag`.

3.1.2 `Letter.java`

La grille du Scrabble est composée de lettres. Une classe `Letter` a été créée pour répondre à ce besoin. La classe `Letter` est un wrapper du type `char`. Dans le Scrabble, les lettres sont toutes en majuscules. Cette classe permet de créer à partir de caractères en majuscules, des lettres bien formées. Cela permet d'être sûr de tout le temps manipuler la même lettre, et de régler le problème de minuscule/majuscule. De plus, tous les caractères n'étant pas des lettres, cette classe permet aussi d'assurer d'avoir une lettre valide.

```
1 public class Letter {
2     public final char chr;
3     public final boolean isJoker;
4 }
```

Cette classe étant un wrapper de `char`, sa complexité est en $\mathcal{O}(1)$.

3.1.3 `LetterCounter.java`

La classe `LetterCounter` est un wrapper sur une `Hashtable<Letter, Integer>`. Cette classe permet, par exemple, de stocker les lettres du chevalet d'un joueur. Le but de cette classe est de savoir quelle lettre contient le chevalet, ainsi que le nombre d'occurrences. Pour cela, elle est implémentée comme une `Hashtable<Letter, Integer>` [11]. Il s'agit d'une table de hachage, qui pour une lettre donnée renvoie son nombre d'occurrences. Étant donné que cette classe est un wrapper, des méthodes telles que l'ajout d'occurrences, la suppression, le nombre d'occurrences, etc. ont été ajoutées. Grâce à ces méthodes, l'intégrité est garantie. De plus, il s'agit d'un dictionnaire (au sens de structure de données), sa complexité est $\mathcal{O}(1)$ pour l'accès et l'écriture des données.

```
1 public class LettersCounter {
2     /**
3      * dictionary type like:
4      * A: 0
5      * B: 45
```

```
6       * how many A in
7       */
8     private final Hashtable<Letter, Integer> table;
9 }
```

3.1.4 `Grid.java`

La classe Grid permet de stocker le plateau de jeux.
Elle contient une matrice de lettres placées : `Letter[][] placedLetters`.
Cette classe contient aussi une autre matrice permettant de connaître les multiplicateurs de points en fonction des positions sur la grille :
`Hashtable<GridPos, ScoreMultiplier> multiplicators`. Cette table de hachage permet de savoir pour une `GridPos` (x, y) donnée quel sera le multiplicateur à appliquer.
Les multiplicateurs déjà utilisés sont marqués dans
`boolean[][] multiplicatorsUsed`. En fonction de la langue et de l'activation ou non du mode super Scrabble, chaque lettre vaut un nombre de points différent. Les points associés aux lettres sont stockés dans `LettersPoints pointsPerLetter`.
La taille de la grille est stockée dans `int size`.

Les attributs complexes ont été expliqués ci-dessus, mais cette classe en comporte d'autres :

```
1 public class Grid {
2     public final int size;
3     private Letter[][] placedLetters;
4     private final Hashtable<GridPos, ScoreMultiplier>
          multiplicators;
5     private final boolean[][] multiplicatorsUsed;
6     private final LettersPoints pointsPerLetter;
7     private final GameModel gameModel;
8     private final boolean isSuperScrabble;
9 }
```

Cette classe implémente une grille, c'est-à-dire une matrice bidimensionnelle. L'accès à un élément de cette matrice se fait en $\mathcal{O}(1)$ grâce à l'implémentation sous-jacente de Java.

3.1.5 `History.java` et `HistNode.java`

L'Historique est une pile implémentée par une liste doublement chaînée classique. Les nœuds de cette liste, `HistNode`, contiennent les données nécessaires

pour pouvoir effectuer une action de la partie.

Cette structure s'avère adaptée pour nos besoins relatifs au jeu. En effet, nous ajoutons seulement des éléments à la fin de l'historique, après que le joueur ait effectué son coup, la pile est donc adaptée. Quand il s'agit de retrouver des éléments de l'historique, nous avons trois cas d'usages :

— Défaire un coup (undo) : On retrouve l'élément précédant la tête de pile $\mathcal{O}(1)$

— Refaire un coup (redo) : On retrouve l'élément suivant la tête de pile $\mathcal{O}(1)$

— Écrire l'historique dans un fichier : On parcourt la pile de haut en bas O(n)

Lorsque l'on défait un coup, on fait reculer la tête de pile, sans détacher les actions suivantes 3. On peut donc défaire plusieurs coups, et venir les refaire après car ils sont toujours disponibles dans la pile. Cependant, cette structure de données n'est pas une solution parfaite pour l'historique. À tout moment de la partie, nous avons seulement un chemin de changement pour l'utilisateur. Concrètement, si un utilisateur revient en arrière dans l'historique, puis décide de jouer un nouveau coup, tout l'historique suivant original est abandonné. Une solution aurait été d'implémenter un arbre au lieu d'une pile.

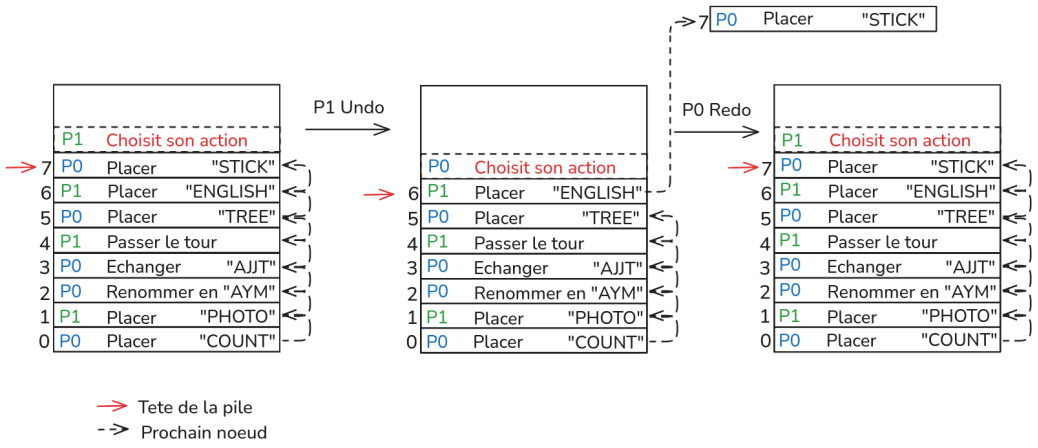


FIGURE 3 – Undo puis redo sur l'historique d'une partie (les références vers le nœud précédent sont omises)

```
public class History<T> {
    private int nextNodeID;
    private HistNode<T> currentNode;
    private final HistNode<T> nullNode;
}
```

```
6
7  class HistNode<T> {
8      private int nodeID;
9      private HistNode<T> prev;
10     private HistNode<T> next;
11     private T value;
12 }
```

3.2 Algorithmes

3.2.1 `Parser`

L'objectif du parser est d'analyser un fichier de sauvegarde du scrabble et d'en sortir un modèle utilisable, tout en jetant des erreurs indiquant la ligne concernée à l'utilisateur. Pour cela, nous avons implémenté notre propre solution de parsing. Le parsing est un sujet qui a vu beaucoup de recherche dans le domaine de l'informatique car c'est un problème commun en particulier pour l'analyse de code source dans le cadre de compilateur. Nous nous sommes inspirés de l'organisation des parsers formels pour mettre en forme notre solution, en particulier avec l'élaboration de différents types de tokens. Cependant, notre solution reste très simpliste, car nous faisons toute l'analyse en une seule passe, et au lieu de générer un arbre de syntaxe (formellement un AST), nous remplissons les données de notre modèle de jeu. Cette approche est inspirée par des ressources trouvées en ligne tel que [1] et semble adaptée pour notre problème qui reste assez simple (analyse de fichier de sauvegarde).

3.2.2 `Tokens`

Notre parser ne traite pas directement de caractères ou de lignes du fichier d'entrée, il agit sur des éléments abstraits, des tokens. Ces tokens représentent une valeur "terminale" du fichier de sauvegarde, c'est-à-dire une valeur qui ne peut pas être subdivisée en éléments plus petits. Cette abstraction permet de diminuer la complexité des algorithmes du parseur, car nous n'avons pas besoin de nous soucier du placement exact des caractères dans le fichier pour en extraire le contenu.

Liste des Tokens		
Type	Description	Exemple
Ponctuation	Un de "`.{}<>[],;$`"	;
Opérateur	=	=
Nombre	1 a 8 chiffres	30
Nombre Négatif	1 a 8 chiffres, précédé de "-"	-30
Long	9+ chiffres	111111111
Long Négatif	9+ chiffres, précédé de "-"	-111111111
Identifiant	`[A-Z][0-z]+`	Language
String	"`.+`"	"mcts"
Move	`[a-z][0-9]+[a-z]`	h8h

Les expressions régulières sont seulement utilisées à titre indicatif dans la description des tokens.

Nous pouvons donc ensuite décrire la série de tokens souhaitée au parseur, puis former un modèle à partir de ces valeurs.

Algorithme 1 : Parsing du modèle serializé

Entrées : $tokenizer$
Sorties : $modele$
Fonction `EstIdentifiant(`$String$`)` **:**
    $prochainToken \leftarrow tokenizer.prochain()$
    Retourner prochainToken.type est "Identifiant" et
     prochainToken.value est String
Fonction `EstCleGame()` **:**
    $prochainToken \leftarrow tokenizer.prochain()$
    Retourner prochainToken.type est "Identifiant" et
     prochainToken.value $\in$ clesGame
Fonction `EstIdAction()` **:**
    $prochainToken \leftarrow tokenizer.prochain()$
    Retourner prochainToken.type est "Identifiant" et
     prochainToken.value $\in$ typesAction
Fonction `Parser(`$tokenizer$`)` **:**
    $prochainToken \leftarrow tokenizer.prochain()$
    tant que $prochainToken \neq EOF$ **faire**
        PasserPonctuation('<')
        si *EstIdentifiant("GAME")* **alors**
            tant que *EstCleGame()* **faire**
                passerOperateur()
                $modele \leftarrow parserVarCourante()$
        si *EstIdentifiant("HISTORY")* **alors**
            $prochainToken \leftarrow tokenizer.prochain()$
            tant que *prochainToken.type est "Nombre"* **faire**
                PasserPonctuation('.')
                si *EstIdAction()* **alors**
                    $modele \leftarrow parserTourCourant()$
                PasserPonctuation(';')
        PasserPonctuation('>')
    retourner $modele$

Les méthodes finales de l'algorithme de parsing 1 ne sont pas développées ici car peu pertinentes d'un point de vue algorithmique. La fonction `PasserPonctuation` vérifie que le prochain caractère est bien la ponctuation attendue et avance le curseur sans utiliser la valeur. Il est notable que nous sérialisons peu d'informations dans les fichiers de sauvegarde. En effet, au lieu de récupérer les états complets de

la partie depuis le fichier, nous récupérons l'état initial au début de la partie, ainsi qu'une liste des coups, que nous rejouons 1 à 1 pour revenir à l'état décrit par le fichier. Cette manière de retrouver les états est possible car nous invoquons tous nos appels aléatoires (utilisés dans la pioche) à l'aide de l'algorithme pseudo-aléatoire de Java instancié avec une seed, qui elle est sérialisée. Cette seed est retrouvée, parmi d'autres propriétés dans `parserVarCourante()` et les coups sont rejoués lors de `parserTourCourant()`.

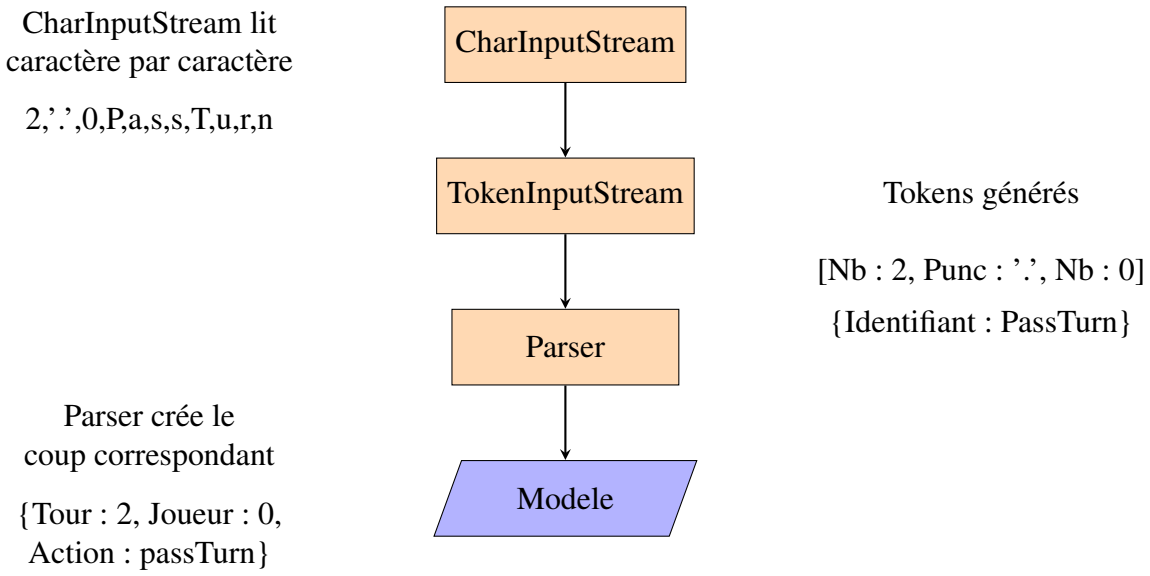


FIGURE 4 – Schéma du parsing de "2. 0 PassTurn"

La complexité du parser est en $\mathcal{O}(nb_char_in_file)$ avec `nb_char_in_file` étant le nombre de caractères présents dans le fichier donné.

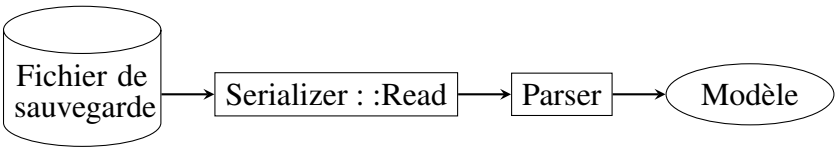
3.2.3 `Serializer`

Le `Serializer` est la classe qui s'occupe de sauvegarder l'état d'une partie dans un fichier ainsi que de le recharger. Pour ce qui est de charger l'état de la partie, elle appelle le parseur décrit précédemment. Quand il s'agit de sauvegarder un modèle vers un fichier, on utilise notre structure de données `History`.

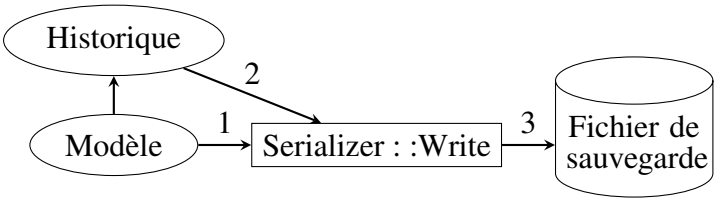
Algorithme 2 : Sérialisation de l'historique vers String

Entrées : $history$
Sorties : $texteSerialise$
$noeud \leftarrow history.currentNode$
$pileActions \leftarrow []$
tant que $noeud \neq debut$ **faire**
    pileActions.empiler(noeud.action)
    $noeud \leftarrow noeud.precedent$
$texteSerialise \leftarrow$ ""
pour tout $action \in pileActions$
    $texteSerialise \leftarrow texteSerialise + action.stringSerialise$
retourner $texteSerialise$

La complexité asymptomatique de notre algorithme de sérialisation de l'historique 2 est linéaire selon le nombre d'actions jouées dans la partie. Mais on constate que l'on itère tout de même deux fois sur les actions. Une fois pour récupérer les actions dans l'ordre, une deuxième fois pour produire le string. Nous adoptons cette approche car cela nous permet de créer le String dans l'ordre d'arrivée des actions, en effet itérer une seule fois et récupérer les actions dans l'ordre inverse nous demanderait de sérialiser en ajoutant à la position 0 du string, ce qui est une opération coûteuse.



Lire depuis le fichier de sauvegarde



Ecrire vers le fichier de sauvegarde

FIGURE 5 – Schéma de flux de Serializer

3.2.4 `placeLetters()`

Explication en pseudo-code de haut niveau de la méthode
`public List<OwnedWord> placeLetters(PlaceWordAction pwa)`

de `Grid.class` liée à la `Feature 32` :

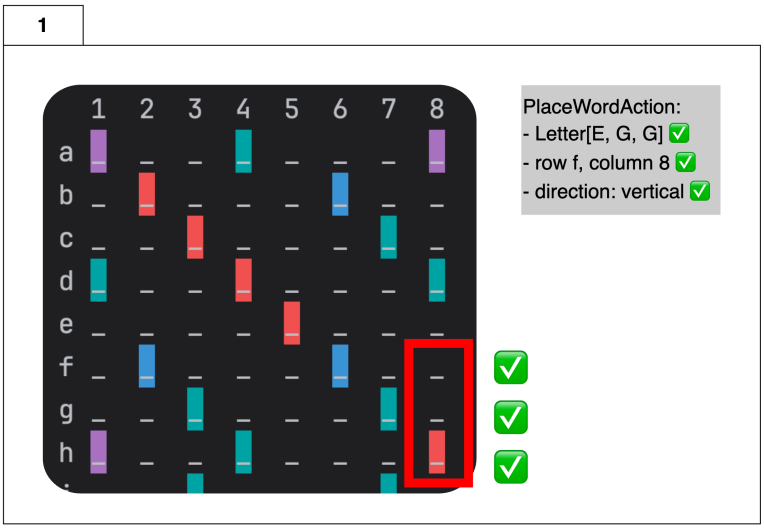
La complexité au pire de cet algorithme est en $\mathcal{O}(grid_width \times grid_heigt)$, avec $grid_width = grid_height$ étant la taille de la grille, c'est-à-dire 15 en mode normal, et 21 en mode super-scrabble.

Algorithme 3 : List<OwnedWord> placeLetters(PlaceWordAction)

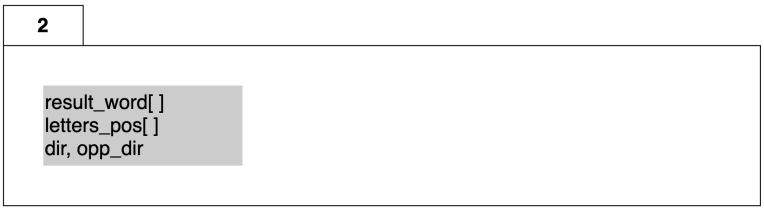
Data : PlaceWordAction {
| tableau de Letter a placer sur la grille
| ligne x et colonne y où placer les lettres
| direction de placement
}
Result : List<OwnedWord> {
| liste de mots formés avec leurs positions sur la grille
}
Vérification des prérequis :
| Si aucune lettre n'est à placer, renvoyer une erreur
| Si la position de départ n'est pas dans les limites de la grille, renvoyer une erreur
| Si la case de départ n'est pas vide, lever une erreur
Création des variables :
| Liste de mots résultants formés
| Liste pour stocker les futurs positions des lettres à placer
| Direction du mot à placer et sa direction opposée
Calculer les positions des futurs lettres à placer :
| Pour chaque lettre à placer, calculer et enregistrer sa futur position;
`/* Sert quand des lettres sont déjà présentes sur la grille */`
Construire le mot principal :
| Déterminer les positions de début et de fin du mot principal et le construire
Construire les mots auxiliaires :
| Pour chaque lettre, vérifier si elle forme un mot avec les lettres adjacentes et le construire si c'est le cas
Vérifier la validité des mots :
| S'assurer que tous les mots construits sont présents dans le dictionnaire, sinon renvoyer une erreur
Contraintes de placements :
| Vérifier la règle du premier placement sur la grille :
| | Si c'est le premier placement sur la grille de la partie, vérifier qu'il touche le centre
| Vérifier les placements adjacents :
| | S'assurer que les mots sont placés à côté d'autres mots
Placer les lettres sur la grille :
| Pour chaque lettre, placer la lettre à sa position calculée
Retourner la liste des mots construits :
| Retourner la liste des mots qui ont été placés sur la grille

Nous allons dérouler l'algorithme précédent 3, et expliquer les étapes. On souhaite placer `EGG` en `f8v`. La grille affichée sur les images suivantes est coupée pour se focaliser sur la partie intéressante, et ne pas afficher une grille de 15×15 100% vide.

La première étape de l'algorithme est de vérifier les conditions d'entrée, c'est-à-dire que le tableau de lettres à poser ne soit pas vide, que la position soit dans la grille et qu'il n'y ait pas une autre lettre sur la position de départ. L'algorithme `placeLetters()` pose des lettres sur le plateau pour former un mot, il ne faut pas le confondre avec l'entrée de la commande de l'utilisateur, qui lui souhaite poser un mot alors que cet algorithme est fait pour placer les lettres, non pas un mot entier.

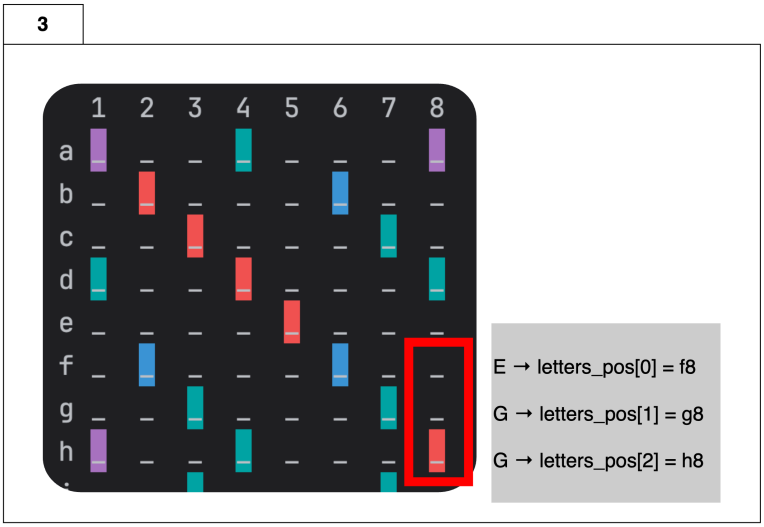


Nous créons ensuite des variables, telles que la liste des mots formés `result_word` et les futures positions des lettres à placer `letters_pos`. Comme nous plaçons des lettres sur une grille qui peut déjà en contenir, les lettres que nous souhaitons placer peuvent être séparées par d'autres lettres déjà placées sur la grille, nous avons besoin de savoir où nos futures lettres à placer seront mises.

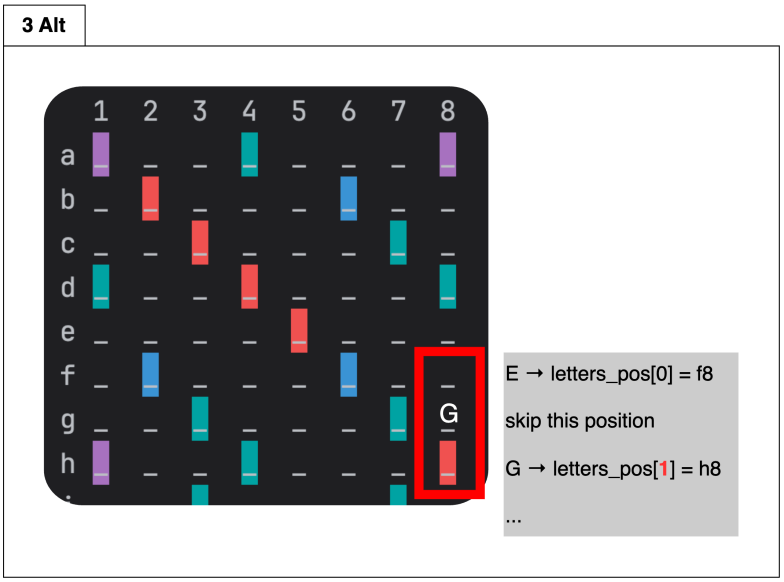


Nous allons maintenant calculer les futures positions de nos lettres à placer.

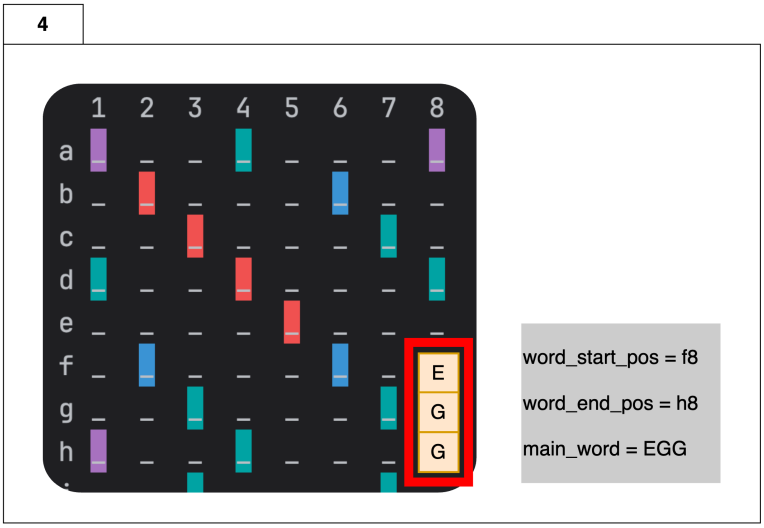
Pour cela, dans le cas 3, les lettres à placer seront à la suite les unes des autres, vu qu'il n'existe pas de lettres sur la grille sur le chemin de nos nouvelles lettres.



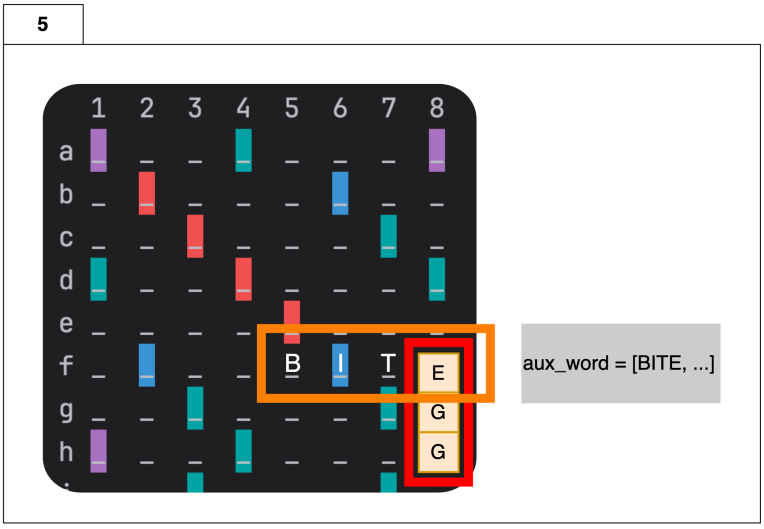
Dans ce scénario alternatif du cas 3, il y a une lettre sur le chemin de nos lettres à placer. Nous en tenons compte, et décalons la position de notre deuxième lettre pour tenir compte de la lettre déjà présente sur la grille.



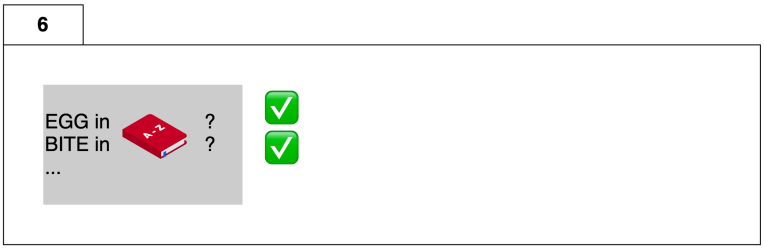
Nous savons maintenant où placer nos lettres, et quels sont les lettres déjà présentes sur la grille. Nous allons calculer la position de départ et la position finale du mot que nous venons de placer. Nous stockons ceci dans des variables.



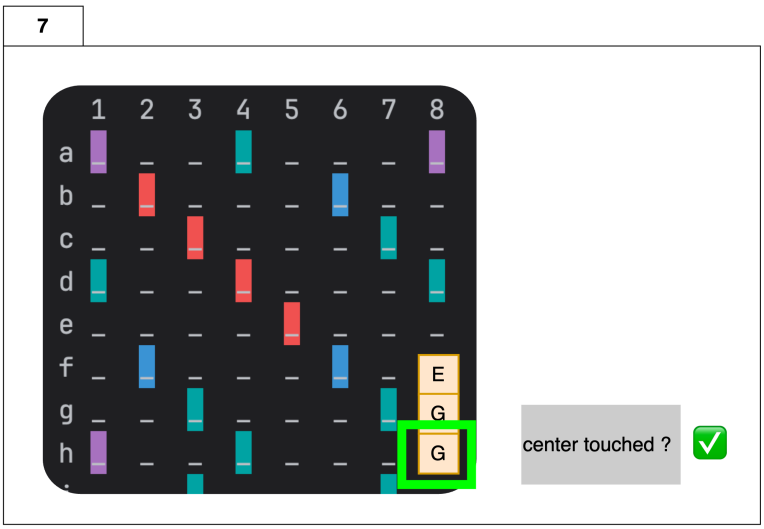
Nous vérifions si le fait de placer nos lettres a pu former d'autres mots auxiliaires. Prenons pour exemple le mot en blanc `BIT` comme déjà placé sur la grille. En rajoutant nos lettres, nous formons donc un nouveau mot, qui n'est pas le mot principal, mais un auxiliaire, il s'agit du verbe `BITE`.



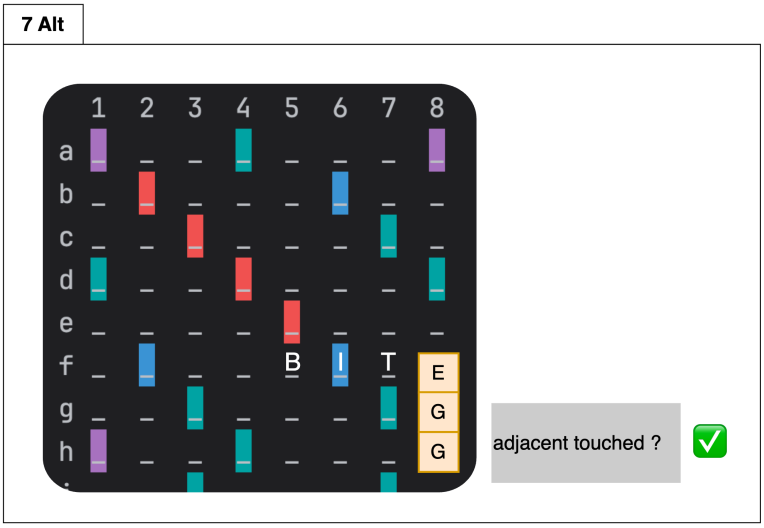
Nous vérifions ensuite que tous les mots créés appartiennent au dictionnaire.



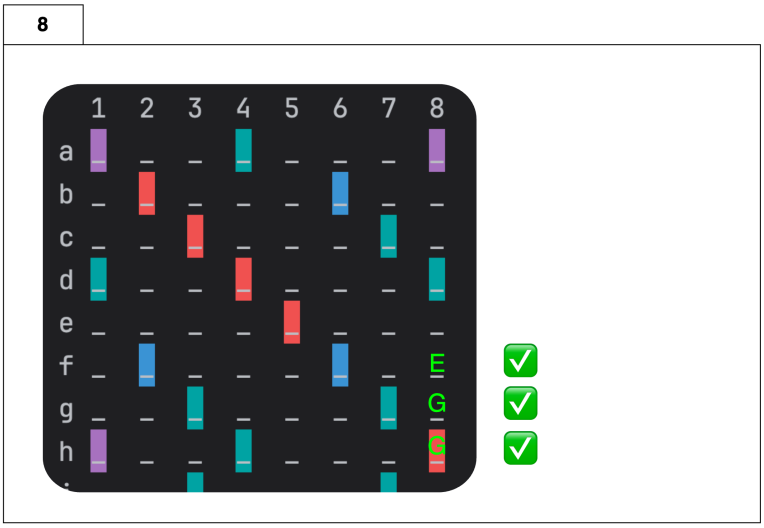
Ensuite, nous vérifions les contraintes de placement. En début de partie, le premier joueur doit placer son mot de sorte à ce qu'il touche le centre de la grille (`h8` en mode normal, `k11` en super scrabble). Nous vérifions cette condition s'il s'agit du tout premier placement de lettres.



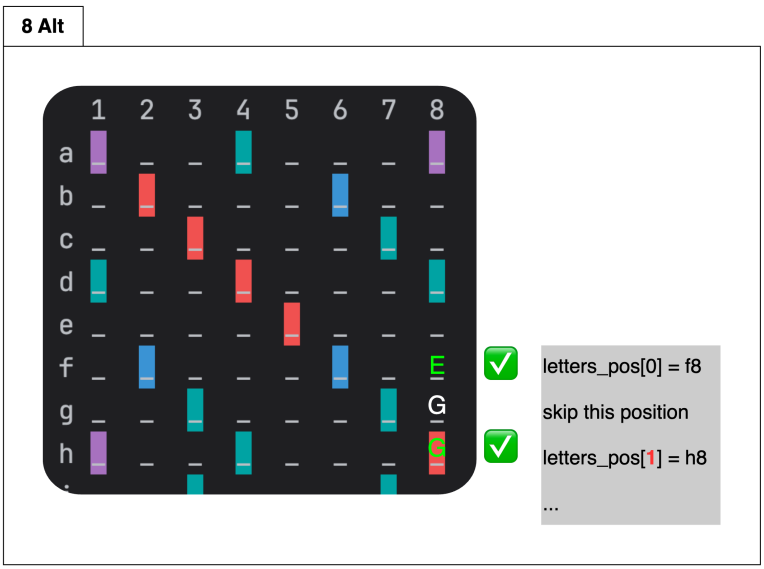
Si il ne s'agit pas du premier coup, nous vérifions que les lettres placées touchent un mot déjà présent sur la grille.



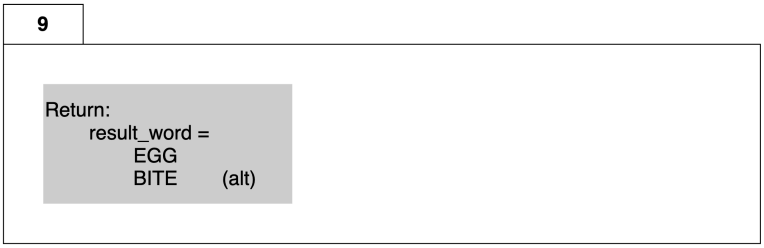
Nous pouvons placer les lettres sur la grille.



Dans ce scénario alternatif, où nous plaçons nos lettres avec une autre lettre déjà présente sur la grille, le tableau de position `letters_pos` est là pour nous indiquer où placer les lettres. De même pour le cas précédent, mais ce tableau illustre son utilité dans ce cas présent.



Après toutes ces étapes de vérification, de calcul, et de placement nous pouvons retourner la liste des mots formés par le placement des lettres. En fonction du scénario standard ou alternatif `Alt`, nous retournons la liste `result_word` avec le mot `[EGG]` ou la liste avec les deux mots `[EGG, BITE]`.



4 Architecture

4.1 Architecture

Nous avons choisi une architecture `Model-View-Presenter` (MVP [12]) pour organiser les modules du projet. Cette architecture a été choisie car elle pose les lignes directrices sur la gestion des flux entre les classes. De plus, elle est optimisée pour la gestion d'affichage en console.

Le `Model` contient les différentes structures de données, et autres éléments logiques relatifs aux contraintes du jeu. Les éléments de cette couche correspondent

à ce qui est nécessaire à la partie (joueurs, lettres, multiplicateur, grille, etc).

La `View` est constituée de tous les éléments visibles par l'utilisateur. Il s'agit d'afficher du texte, des couleurs, des scores, des demandes d'interaction, des boutons, etc. Elle est implémentée en deux classes, `ViewCLI` pour la partie CLI et `ViewGUI` pour la partie GUI.

4.1.1 `ViewCLI` sur MVP

L'architecture MVP est adaptée à l'affichage en console, et nous l'utilisons donc pour `ViewCLI`.

Le `Presenter` reçoit les demandes de l'utilisateur depuis la `View`, les traite, demande les modifications au `Model`, puis récupère les nouvelles informations contenues dans le `Model`, et donne ces nouvelles informations actualisées à la `View`.

Pour la gestion des flux, voir 6 :

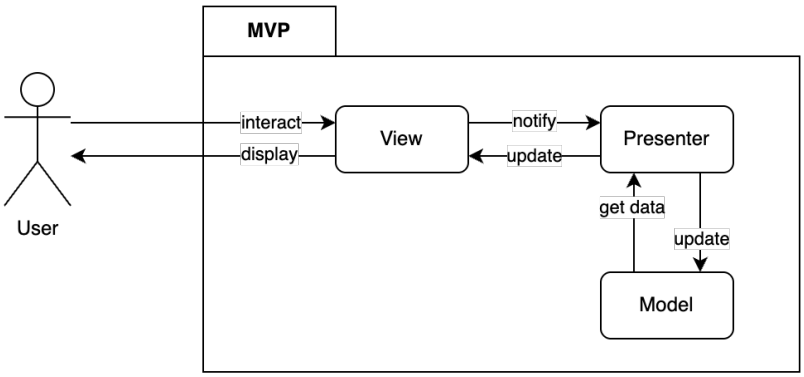


FIGURE 6 – MVP scheme

Nous allons détailler comment fonctionnent les trois composants qui sont le `View`, le `Presenter` et le `Model` grâce à leur implémentation dans le code. Nous verrons seulement trois grandes classes, car les autres classes sont des objets qui contiennent en général des données, et ne sont pas liés à l'architecture. Elles sont utilisées par l'architecture mais n'ont pas de rôle dans son fonctionnement. Par exemple, la classe `Configuration` sert à stocker des données de configuration, comme le nombre de joueurs, le mode de jeu, etc. Elle n'a donc pas de rôle dans l'architecture, et il en va de même pour les autres classes qui ne sont pas présentes dans les schémas architecturaux.

L'image 7 représente l'implémentation du `MVP`, ainsi que le cheminement des informations, sachant que la syntaxe sur le schéma suivant n'est pas strictement la même que celle du code pour simplifier la lecture. Lorsque l'utilisateur interagit avec la `ViewCLI`, par exemple en envoyant dans la console une commande pour se renommer, le `ViewCLI` va avertir le `GamePresenter` de la demande qu'il a reçue de l'utilisateur via une `GameAction`. Une `GameAction` est une classe contenant le numéro du joueur, ainsi que sa demande, dans cet exemple, le fait de changer de nom. Le `GamePresenter` va regarder quelle est la nature de cette `GameAction` et en fonction de celle-ci, demander la modification des données dans le `GameModel`. Le `GameModel` va traiter la modification, puis le `GamePresenter` va récupérer les nouvelles données mises à jour du `GameModel` puis les envoyer au `ViewCLI` pour qu'il puisse afficher les données actualisées à l'utilisateur.

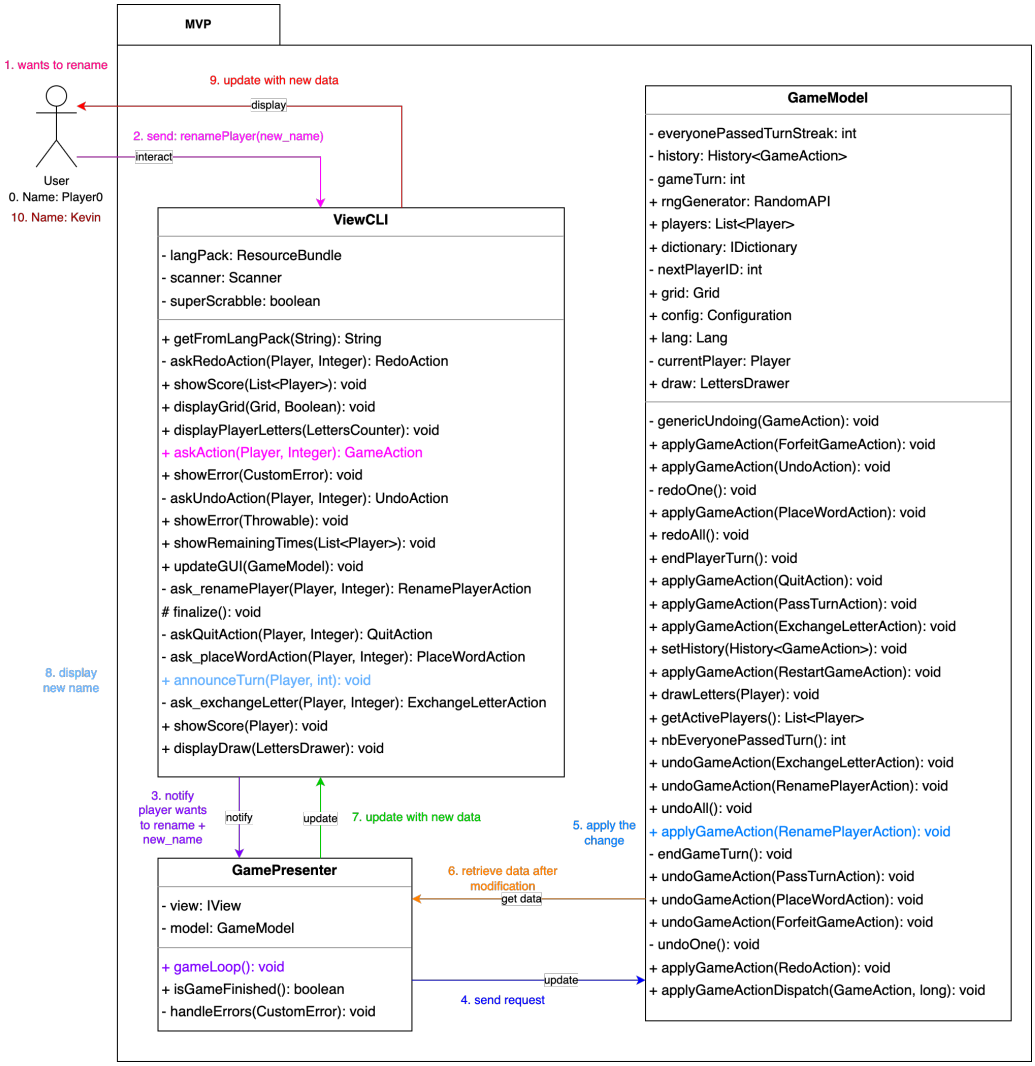


FIGURE 7 – MVP implemented

4.1.2 `ViewGUI` sur VM

Nous avons essayé d'implémenter le `ViewGUI` sur la même architecture MVP, mais dû à des problèmes d'envoi de données entre le thread contenant `JavaFX` et celui contenant le `Presenter`, nous avons abandonné cette architecture pour passer à une plus simple. Étant donné que `JavaFX` est sur un autre thread, la communication était asynchrone. Nous n'avons pas réussi à pallier ce problème et nous avons décidé de partir sur une architecture synchrone et donc plus simple.

L'interface graphique repose sur l'architecture `View Model` 8. Le `Model` reste le même que pour le CLI. La différence est qu'il n'y a pas de `Presenter` au milieu des échanges. La `View` et le `Model` sont sur un seul thread, ce qui résout le problème évoqué précédemment.

Quand l'utilisateur initie une action sur la `View`, celle-ci va faire une modification sur le `Model`, puis récupérer les nouvelles informations contenues dans le `Model` pour les afficher.

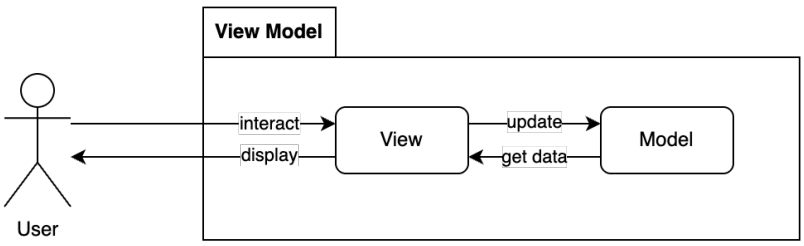


FIGURE 8 – View Model scheme

L'image 9 représente l'implémentation du `MV`, ainsi que le cheminement des informations, sachant que la syntaxe sur le schéma suivant n'est pas strictement la même que celle du code pour simplifier la lecture. Lorsque l'utilisateur interagit avec le `ViewGUI`, par exemple en cliquant sur le bouton pour se renommer, le `ViewGUI` va créer une `GameAction` puis il va regarder quelle est la nature de cette `GameAction` et en fonction de celle-ci, demander la modification des données dans le `GameModel`. Le `GameModel` va traiter la modification, puis le `ViewGUI` va récupérer les nouvelles données mises à jour du `GameModel`, puis afficher les données actualisées à l'utilisateur.

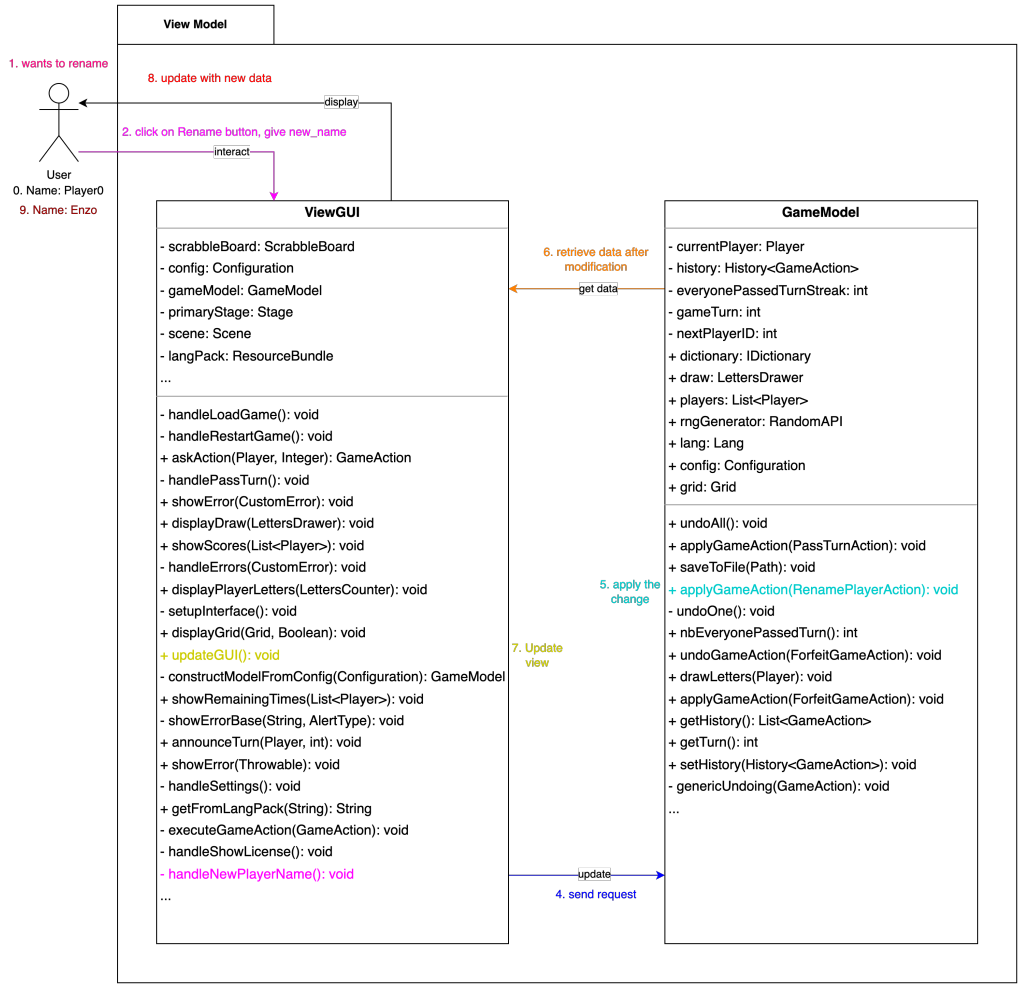


FIGURE 9 – MV implemented

4.2 Gestion des erreurs

Chaque classe, ainsi que ses méthodes, renvoie des `CustomError` lors de problèmes rencontrés. Toutes les classes sont appelées par le `Presenter` qui les coordonne. Le `Presenter` contient la `game loop` qui s'occupe de la gestion du jeu, et qui récupère les erreurs soulevées de toutes les classes du programme. Lorsqu'une erreur est soulevée, il l'affiche à l'utilisateur. De ce fait, toute erreur pouvant arriver lors de l'exécution sera rattrapée par la `game loop` et affichée, ne provoquant pas un arrêt brutal du programme.

4.3 Dépendances du projet

Le projet est codé en Java version `17.0.13`, et est compilé à l'aide du système de build Maven. Afin de respecter les spécifications du projet, un certain nombre de dépendances sont ajoutées :

— `JUnit` : Framework de test, utilisé pour les différents scénarios de test du code.

— `Jacoco` : Librairie de couverture de code, utilisée en conjonction avec JUnit pour certifier le pourcentage de code source testé.

— `Commons-cli` : API d'analyse d'options de commandes. Utilisé pour interpréter les options passées en ligne de commande.

— `JavaFx` : Framework d'interface utilisateur, utilisé pour la partie interface graphique.

— `Spotless` : Outil d'analyse de code permettant de vérifier la conformité du formatage avec le style de codage de Google.

5 Performances et limitations

5.1 Choix de conception et limitations observées

Le projet Scrabble a été élaboré en se focalisant sur trois aspects principaux : la clarté, grâce au format Google Format [5] ; la modularité et la facilité de maintenance du code, avec une grande partie du code documentée.

La modularité du projet est assurée grâce à l'utilisation de l'architecture MVP [12]. Cette structure permet une séparation nette des responsabilités au sein du code, ce qui facilite son développement et sa maintenance. En isolant chaque composant (modèle, vue et contrôleur), nous avons pu garantir que les différentes parties du projet puissent évoluer indépendamment, sans risque de conflit. Cette approche modulaire offre une grande flexibilité, permettant des modifications ciblées et une évolution facile du système sans perturber l'ensemble du projet.

La lisibilité du code a été une priorité, et cela a été rendu possible grâce au Google Format, qui garantit une présentation claire et structurée. De plus, les commentaires sont soigneusement indentés, permettant ainsi de comprendre facilement ce que chaque fonction fait, indépendamment de la taille de l'IDE. Les lignes longues sont également gérées de manière à conserver une lecture fluide et agréable.

Le langage Java, utilisé pour l'implémentation, propose des abstractions de haut niveau et une gestion automatique de la mémoire grâce à un garbage collector [10]. Cela nous a permis de focaliser nos efforts sur l'implémentation du jeu et des règles du Scrabble ainsi que sur la solidité du fonctionnement global.

En pratique, les performances observées sont très satisfaisantes. Le jeu se charge quasiment instantanément sur des machines ordinaires, qu'il soit utilisé via une interface en ligne de commande (CLI). Aucune action, y compris celles qui se rapportent au chargement du dictionnaire officiel du Scrabble (en français et en anglais), à la vérification des mots, à l'évaluation des points ou à la disposition des lettres sur le plateau, n'a montré de délai perceptible. Il convient de noter que, bien qu'aucun instrument de mesure de performance n'ait été intégré dans l'application (tels que des chronomètres ou des analyseurs de performance), l'ensemble du comportement a été évalué empiriquement à travers les interactions utilisateur : les réactions de l'interface sont immédiates, sans latence visible.

Du point de vue de l'interface par rapport à la de jeu, la CLI est stable, réactive et complète, cependant le développement de l'interface graphique nous a posé de nombreux problèmes ayant mené à l'abandon du développement de ces features, notamment à cause de conflits entre le thread principal et le thread JavaFx Application.

En termes de limitation, le projet remplit en grande partie les fonctionnalités spécifiées dans le cahier des charges initial. La partie sur l'intelligence artificielle et l'implémentation d'un joueur artificiel n'a pas été implémentée. Cela représente une piste d'amélioration future, hors du périmètre de cette version. Aucune limitation technique n'empêche a priori l'évolution du code vers cette fonctionnalité. De plus, suite à un oubli lors du développement, le temps restant pour les joueurs en mode Blitz n'est pas sauvegardé dans le fichier d'historique.

Notre code suit le design pattern architectural MVP. Ce design pattern est très pratique, mais il ajoute quand même des limitations. Si nous voulions dans le futur rajouter des fonctionnalités se rapprochant du cahier des charges actuel, ce design pattern ne bloquerait pas la modification. Il serait même facile d'ajouter ces dites fonctionnalités. Par contre, l'ajout d'un mode de jeu changeant profondément le Scrabble ne serait pas possible sans changer profondément du l'architecture du MVP.

5.2 Analyse des performances

Au-delà des observations qualitatives, une série de micro-benchmarks a été intégrée au projet afin de mesurer les performances des opérations critiques telles que le *load* et le *write* d'une partie, la recherche de mots dans le dictionnaire, le calcul des scores, ou encore le placement de mots sur le plateau.

Tous les tests ont été effectués sur une machine personnelle, et les résultats présentés correspondent à une moyenne sur 100 itérations. Les historiques utilisés pour ces tests sont disponibles dans le dossier `saves/perfs`. Pour exécuter ces tests sur une autre machine, la commande suivante peut être utilisée :

```
mvn -Dtest=PerfTestsMeans test
```

Nom du test	**Détail**	**Temps (ns)**	**Temps ($\lceil t/10^6 \rceil$ ms)**
loadSmallHist		29 039 706	30
loadMediumHist		32 864 802	33
loadLargeHist		30 701 072	31
writeSmallHist		125 478	$<$ 1 ms
writeMediumHist		74 931	$<$ 1 ms
writeLargeHist		249 386	$<$ 1 ms
testScoreCalcEn	small word	8 327	$<$ 1 ms
testScoreCalcEn	medium word	10 583	$<$ 1 ms
testScoreCalcEn	large word	10 084	$<$ 1 ms
testScoreCalcFr	small word	8 206	$<$ 1 ms
testScoreCalcFr	medium word	8 854	$<$ 1 ms
testScoreCalcFr	large word	10 186	$<$ 1 ms
testplaceWord	empty grid	18 786	$<$ 1 ms
testplaceWord	occupied grid	16 783	$<$ 1 ms
searchWordEn	existent	2 061	$<$ 1 ms
searchWordEn	non-existent	1 776	$<$ 1 ms
searchWordFr	existent	1 955	$<$ 1 ms
searchWordFr	non-existent	1 753	$<$ 1 ms

TABLE 1 – Résultats des micro-benchmarks sur 100 itérations

Les tests de type `load/write` utilisent trois tailles d'historique :

— **Small** : 3 tours, 4 mots placés.

— **Medium** : 6 tours, 10 mots placés (plus de joueurs).

— **Large** : 8 tours, 20 mots placés (limite atteinte sur la pioche).

Les tests `testScoreCalcXXX` mesurent uniquement le temps de calcul des scores, selon la taille du mot et la langue. Les tests `searchWordEn/Fr` évaluent

quant à eux le temps de recherche d'un mot dans le dictionnaire, qu'il existe ou non. Ces derniers s'appuient principalement sur la structure de *Hashtable* standard de Java [11].

Les résultats obtenus montrent des temps d'exécution très faibles sur l'ensemble des cas testés. Aucun ralentissement significatif n'a été observé, et aucun *bottleneck* identifié dans les opérations courantes. Le projet offre donc des performances solides et constantes, même sur des historiques relativement longs.

6 Critiques et perspectives

Grâce à la gestion rigoureuse et à la vision claire du code du projet, nous avons pu maintenir une architecture cohérente tout au long du développement. Néanmoins, certaines décisions et ajustements structurels ont dû être pris de manière impromptue au fil de l'avancement du projet. Par ailleurs, une segmentation plus fine de certaines parties du code aurait sans doute amélioré la lisibilité et la compréhension globale. Prendre davantage de temps pour faire des bilans réguliers et, si nécessaire, repenser l'organisation aurait été bénéfique pour un projet de cette ampleur.

Le projet, bien qu'opérationnel et jouable dans son état actuel, reste incomplet sur certains aspects, notamment l'absence d'une intelligence artificielle. Toutefois, certaines fonctionnalités que nous souhaitions intégrer se sont révélées complexes à mettre en œuvre et non prioritaires à ce stade. C'est le cas, par exemple, d'un mode multijoueur sur plusieurs machines ou d'un système de classement basé sur les scores. Une sous-estimation de certaines tâches a engendré des retards en fin de parcours. Nous avons donc fait le choix de privilégier un code stable, clair et sans bugs, plutôt que de multiplier les fonctionnalités incomplètes ou précipitées. Ce manque d'anticipation souligne l'importance d'une meilleure planification pour les projets à venir. Enfin, l'ajout d'une intelligence artificielle et de fonctionnalités avancées constitue une piste d'amélioration majeure pour une version future du projet.

En terme de perspective futur, si nous avions six mois de développement en plus, nous aurions développé un joueur en intelligence artificielle. Nous aurions fait intérêt sur plusieurs versions, en commençant par un MiniMax. Ensuite, comme demandé dans le cahier des charges, nous aurions pu faire l'algorithme de Monte-Carlo. Pour terminer, on serait très certainement passé sur du NegaMax. Un point important lors du développement d'une intelligence artificielle est l'heuristique. Elle nous sert à évaluer à quel point un coup est une bonne ou mauvaise action. Pour cela, il faut donc trouver des modèles mathématiques qui mesurent

à quel point un coup est bon ou mauvais. Nous aurions donc commencé par une heuristique simple tout en essayant de tendre vers une heuristique beaucoup plus complexe, reflétant plus la complexité du Scrabble. Pour terminer sur l'intelligence artificielle, nous aurions essayé d'aller encore plus loin en entraînant un réseau de neurones tel que StockFish aux échecs.

Une autre partie que nous souhaitions améliorer serait sur le GUI. Nous souhaiterions pouvoir placer les lettres sur la grille du Scrabble, grâce à un drag & drop, ce qui n'est pas le cas actuellement. Ensuite, nous souhaiterions améliorer l'affichage du temps restant des joueurs. Actuellement, le temps restant de nos joueurs est affiché à la fin de son coup. Nous voudrions qu'il soit affiché en temps réel. Pour finir, nous ajouterions des animations pour rendre l'expérience utilisateur meilleure.

7 Bibliographie

La correction linguistique a été effectuée à l'aide de ChatGPT d'OpenAI [9] à travers l'outil Writefull intégré à Overleaf.

Références

[1] M. Bazon. Writing a parser. `https://lisperator.net/pltut/parser/`, 2012-2025.

[2] FFSc - Direction Nationale du Scrabble Classique. Règlement international du scrabble classique. `https://www.ffsc.fr/files/public/fichiers/reglements/classique/Reglement.international.du.Scrabble.classique.pdf`, 2019.

[3] FFSc - Direction Nationale du Scrabble Classique. Histoire du scrabble classique. `https://www.fisf.net/scrabble/decouverte-du-scrabble/historique.html`, 2025.

[4] FFSc - Direction Nationale du Scrabble Classique. Jeu en ligne du scrabble. `https://www.fisf.net/138-non-categorise/36-jeu-en-ligne.html`, 2025.

[5] Google. Java style guide. `https://google.github.io/styleguide/javaguide.html`, 2021.

[6] S. A. Gordon. A faster move generation algorithm. *SOFTWARE—PRACTICE AND EXPERIENCE*, 24(2) :219–232, February 1994.

[7] A. W. A. . G. J. Jacobson. The world's fastest scrabble program. *Programming Techniques and Data Structures*, 31 :572–578, May 1988.

[8] Montaigne Sylvain. Jeu en ligne clicmouse. `https://www.clicmouse.fr/scra/index`.

[9] OpenAI. Correction linguistique effectuée à l'aide de ChatGPT. `https://www.openai.com/chatgpt`, 2025. Correction linguistique réalisée à l'aide de l'outil ChatGPT d'OpenAI.

[10] Oracle. Garbage Collector Implementation. `https://docs.oracle.com/en/java/javase/17/gctuning/garbage-collector-implementation.html`, 2024.

[11] Oracle and/or its affiliates. Java SE 17 Documentation. `https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Hashtable.html`, 2024.

[12] M. Potel. Mvp : Model-view-presenter the taligent programming model for c++ and java. *Taligent Inc*, 20, 1996.

A Besoins réalisés

A.1 Liste des besoins réalisés

TODO: syncro feature 22h terminé plus de new

F1	✓	F21	✓	F41	✓
F2	✓	F22	✓	F42	✓
F3	✓	F23	✓	F43	✓
F4	✓	F24	✓	F44	✓
F5	W15	F25	✓	F45	✓
F6	W15	F26	?	F46	✓
F7	✓	F27	W15	F47	×
F8	W15	F28	✓	F48	Alex, Mathéo, W15
F9	✓	F29	Alex, Mathéo, W15	F49	Alex, Mathéo, W15
F10	✓	F30	✓	F50	Alex, Mathéo, W15
F11	✓	F31	✓	F51	✓
F12	✓	F32	✓	F52	Alex, Mathéo, W15
F13	✓	F33	✓	F53	Alex, Mathéo, W15
F14	✓	F34	✓	F54	×
F15	✓	F35	✓	F55	×
F16	✓	F36	✓	F56	×
F17	✓	F37	✓	F57	×
F18	✓	F38	✓	F58	×
F19	✓	F39	✓	F59	×
F20	✓	F40	✓	F60	×

A.2 Note

Précision sur le pourcentage de coverage : JaCoco donne un coverage de 73% (chiffre non exact, dû au fait que le projet est encore en développement, sert seulement comme indicatif). Or, nous n'avons pas à faire les tests sur la partie GUI. Nous pouvons donc calculer que le coverage du projet sans la partie GUI est de 89%.

Calcul indicatif, valeur non à jour :
$(14072 - 3762)/(14072 - (1744 + 757)) = 89.1\%$
$(total - missing)/(total - totalGUI)$